

Отчёта по лабораторной работе 8

Имитационное моделирование

Нджову Нелиа

Содержание

Список иллюстраций

Список таблиц

1 Цель работы

Цель этой лабораторной работы - Изучить дискретно-событийный подход к имитационному моделированию на примере классической модели распространения инфекции SIR. Реализовать стохастическую дискретно-событийную модель в виде программного комплекса на языке Julia. Провести анализ влияния параметров, сравнить со стохастической и детерминированной версиями, оценить производительность и модифицировать модель.

2 Задание

1. Структура программы
2. Скрипт запуска
3. Анализ чувствительности к параметрам
4. Детерминированная длительность болезни
5. Оценка производительности
6. Сохранение результатов в CSV
7. Добавление демографических событий
8. Вакцинация
9. Модель SEIR

3 Теоретическое введение

Модель SIR - это фундаментальная сегментарная модель в эпидемиологии, которая описывает распространение инфекционного заболевания в замкнутой популяции.

Модель делит все население (N) на три отдельные, взаимоисключающие группы:

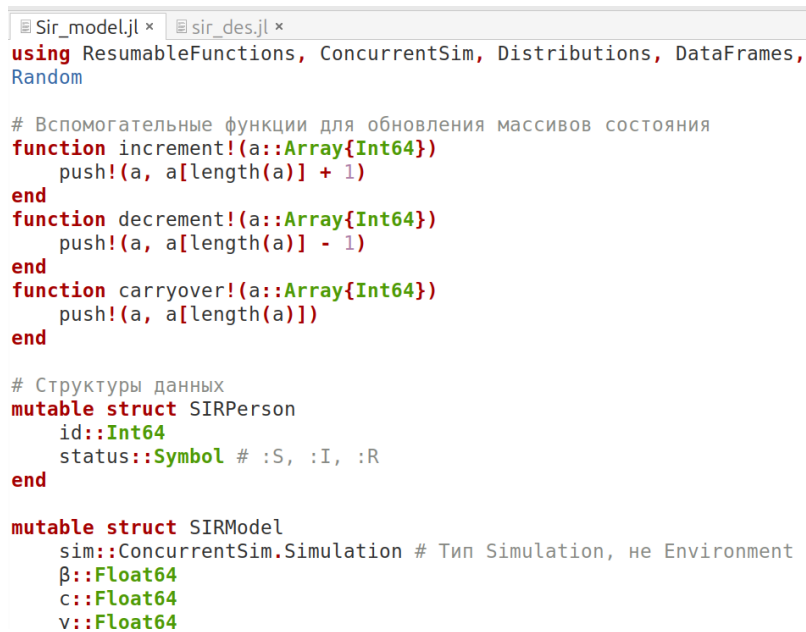
- S (Восприимчивые): люди, которые здоровы, но могут заразиться.
- I (Инфекционные): люди, которые в настоящее время больны и могут передать болезнь восприимчивым людям.
- R (выздоровевшие): люди, которые выздоровели (и считаются невосприимчивыми) или умерли. Они больше не участвуют в процессе передачи.

Общая численность населения постоянна: $N=S+I+R$.

4 Выполнение лабораторной работы

4.1 1. Структура программы

Я создала файл `src/Sir_model.jl` и скопировала заданный скрипт в него. Скрипт реализует ядро модели SIR с дискретными событиями. Он определяет структуры данных, логику поведения агентов и функции управления симуляцией(рис.1)



```
Sir_model.jl * sir_des.jl *
using ResumableFunctions, ConcurrentSim, Distributions, DataFrames,
Random

# Вспомогательные функции для обновления массивов состояния
function increment!(a::Array{Int64})
    push!(a, a[length(a)] + 1)
end
function decrement!(a::Array{Int64})
    push!(a, a[length(a)] - 1)
end
function carryover!(a::Array{Int64})
    push!(a, a[length(a)])
end

# Структуры данных
mutable struct SIRPerson
    id::Int64
    status::Symbol # :S, :I, :R
end

mutable struct SIRModel
    sim::ConcurrentSim.Simulation # Тип Simulation, не Environment
    β::Float64
    c::Float64
    v::Float64
end
```

Рисунок 4.1: Структура программы

Я запускала код и все хорошо сработало.

4.2 2. Скрипт запуска

Я создала файл `scripts/sir_des.jl` и скопировала заданный скрипт в него. Этот скрипт загружает ядро `sir_model.jl`, устанавливает параметры моделирования, запускает моделирование SIR с дискретными событиями и генерирует график динамики S, I и R с течением времени.(рис.2)

```
# ## Модель SIR
#
# Модель создается, процессы активируются, и запуск выполняется.
# Результат преобразуется во фрейм данных. График с кривыми S, I, R
# создается и сохраняется

using DrWatson
@quickactivate "project"
include(scrdir("Sir_model.jl"))
using Random, StatsPlots, DataFrames, CSV

# **Разные параметры β**
# Фиксированные параметры
tmax = 40.0
u0 = [990, 10, 0] # S, I, R
c = 10.0
gamma = 0.25

betas = [0.05, 0.10, 0.15, 0.20, 0.25, 0.30]
results_beta = []

for beta in betas
    p = [beta, c, gamma]
    model = MakeSIRModel(u0, p)
    activate(model)
    sir_run(model, tmax)
    data = out(model)

    peak_I = maximum(data.I) # Вычисляем ключевые метрики
    peak_time = data.t[argmax(data.I)]
    final_R = data.R[end]
```

Рисунок 4.2: Скрипт запуска

Я запускала код и все хорошо сработало, получила график ниже(рис.3)

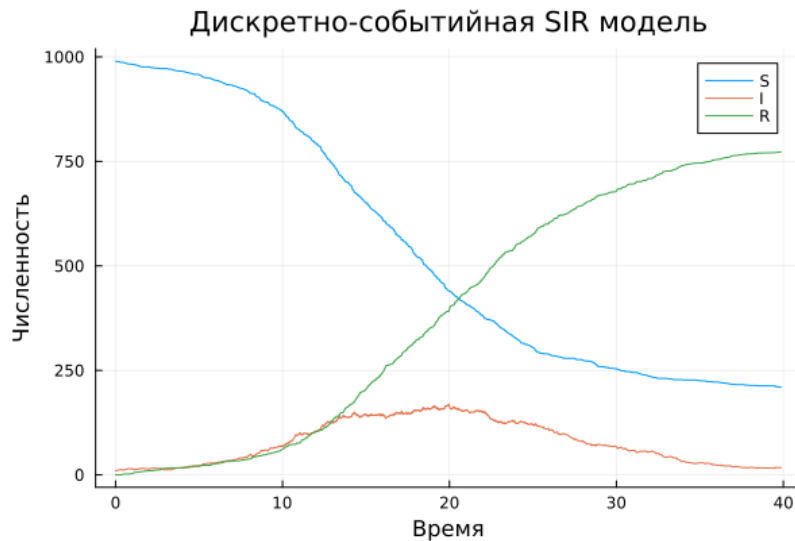


Рисунок 4.3: График

Ниже приведен код, результаты расчетов и графики для кода в файле `scripts/sir_des.qmd`

4.3 Модель SIR

Модель создается, процессы активируются, и запуск выполняется. Результат преобразуется во фрейм данных. График с кривыми S, I, R создается и сохраняется

```
using DrWatson
@quickactivate "project"
include(scrdir("Sir_model.jl"))
using Random, StatsPlots, BenchmarkTools, CSV, Dates
gr(fmt=:png)
```

```
└ Warning: DrWatson could not find find a project file by recursively checking given `dir`
  | (given dir: /home/nelianjovu)
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
```

Plots.GRBackend()

Параметры модели

```
tmax = 40.0
u0 = [990, 10, 0] # S, I, R
p = [0.05, 10.0, 0.25] #  $\beta$ , c,  $\gamma$ 

Random.seed!(1234)
```

TaskLocalRNG()

Запуск модели

```
des_model = MakeSIRModel(u0, p)
activate(des_model)
sir_run(des_model, tmax)
data_des = out(des_model)
```

	t	S	I	R
	Float64	Int64	Int64	Int64
1	0.0	990	10	0
2	0.110851	989	11	0
3	0.249585	988	12	0
4	0.314392	987	13	0
5	0.418738	987	12	1
6	0.457041	987	11	2
7	0.528895	986	12	2
8	0.619502	985	13	2
9	0.657704	984	14	2
10	0.807485	983	15	2
11	0.913646	983	14	3
12	0.976249	982	15	3
13	1.07816	982	14	4
14	1.09161	982	13	5
15	1.1487	982	12	6
16	1.17259	981	13	6
17	1.2789	980	14	6
18	1.30828	980	13	7
19	1.37686	979	14	7
20	1.44574	978	15	7
21	1.45046	977	16	7
22	1.51802	977	15	8
23	1.63961	976	16	8
24	1.71622	976	15	9
25	2.05024	976	14	10
26	2.0729	975	15	10
27	2.20607	974	16	10
28	2.30437	974	15	11
29	2.37251	973	16	11 12
30	2.56655	973	15	12
...	...	...	...	...

Визуализация

```
@df data_des plot(
    :t,
    [:S :I :R],
    labels = ["S" "I" "R"],
    xlab = "Время",
    ylab = "Численность",
    title = "Дискретно-событийная SIR модель",
)
savefig(plotsdir("sir_des.png"))

filename = "sir_$(u0[1])_$(u0[2])_$(p[1])_$(p[2])_$(p[3]).csv"
CSV.write(datadir("sims", filename), data_des)
```

```
"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab08/project/data/sims/sir_990_10_0.05_10.0_0.25.csv"
```

4.4 3. Анализ чувствительности к параметрам

Я создала файл `scripts/sensitivity_beta.jl`, в него, я писала код, похоже с заданный скрипт но с разными значениями β - вероятность передачи инфекции при одном контакте между заразным и восприимчивым (безразмерная величина, от 0 до 1). Я проанализировала пик заражения, время пика и конечное количество выздоровевших людей для каждого значения β

По мере того, как β увеличивает пик заражения, число выздоровевших увеличивается, а время пика уменьшается(рис.4)

```

nelianjovu@Nebby:~/work/study/2026-1/2026-1==study--simulationmod/2026-1--study-
simulationmod/labs/lab08/project$ julia --project=. scripts/sensitivity_beta.jl
β = 0.05 (R₀ = 2.0): Пик I = 178.0, Время пика = 18.0, Финальное R = 769.0
β = 0.1 (R₀ = 4.0): Пик I = 379.0, Время пика = 7.6, Финальное R = 980.0
β = 0.15 (R₀ = 6.0): Пик I = 548.0, Время пика = 4.7, Финальное R = 995.0
β = 0.2 (R₀ = 8.0): Пик I = 608.0, Время пика = 4.3, Финальное R = 998.0
β = 0.25 (R₀ = 10.0): Пик I = 652.0, Время пика = 3.1, Финальное R = 999.0
β = 0.3 (R₀ = 12.0): Пик I = 729.0, Время пика = 3.0, Финальное R = 1000.0

```

Рисунок 4.4: Анализ разных параметров β

Я создала файл `scripts/sensitivity_c.jl`, в него, я писала код, похоже с заданный скрипт но с разными значениями c - среднее число контактов (достаточно тесных для передачи инфекции) одного человека в единицу времени. Я проанализировала пик заражения, время пика и конечное количество выздоровевших людей для каждого значения c

По мере того, как c увеличивает, пик заражения и число выздоровевших увеличивается, а время пика уменьшается (рис.5)

```

nelianjovu@Nebby:~/work/study/2026-1/2026-1==study--simulationmod/2026-1--study-
simulationmod/labs/lab08/project$ julia --project=. scripts/sensitivity_c.jl
c = 5 (R₀ = 1.0): Пик I = 31.0, Время пика = 36.4, Финальное R = 161.0
c = 10 (R₀ = 2.0): Пик I = 199.0, Время пика = 22.7, Финальное R = 767.0
c = 15 (R₀ = 3.0): Пик I = 342.0, Время пика = 11.8, Финальное R = 957.0
c = 20 (R₀ = 4.0): Пик I = 370.0, Время пика = 9.8, Финальное R = 970.0
c = 25 (R₀ = 5.0): Пик I = 465.0, Время пика = 6.5, Финальное R = 991.0
c = 30 (R₀ = 6.0): Пик I = 563.0, Время пика = 4.5, Финальное R = 995.0
nelianjovu@Nebby:~/work/study/2026-1/2026-1==study--simulationmod/2026-1--study-
simulationmod/labs/lab08/project$

```

Рисунок 4.5: Анализ разных параметров c

Я создала файл `scripts/sensitivity_gamma.jl`, в него, я писала код, похоже с заданный скрипт но с разными значениями γ - скорость выздоровления (доля инфицированных, выздоравливающих в единицу времени). Я проанализировала пик заражения, время пика и конечное количество выздоровевших людей для каждого значения γ

По мере того, как γ увеличивает, пик заражения и число выздоровевших уменьшается, а время пика, сначала увеличивается а потом уменьшается (рис.6)

```

nelli@jovv@hebb:~/work/study/2026-1/2026-1=study--simulationmod/2026-1--study--simulationmod/labs/lab08/project$ julia --project=. scr
pts/sensitivity_gamma.jl
γ = 0.05 (болезнь = 20.0 дн): Пик I = 664.0, Время пика = 16.1, Финальное R = 907.0
γ = 0.1 (болезнь = 10.0 дн): Пик I = 497.0, Время пика = 15.0, Финальное R = 979.0
γ = 0.125 (болезнь = 8.0 дн): Пик I = 393.0, Время пика = 17.4, Финальное R = 975.0
γ = 0.1667 (болезнь = 6.0 дн): Пик I = 310.0, Время пика = 15.2, Финальное R = 924.0
γ = 0.2 (болезнь = 5.0 дн): Пик I = 252.0, Время пика = 14.5, Финальное R = 901.0
γ = 0.25 (болезнь = 4.0 дн): Пик I = 182.0, Время пика = 15.3, Финальное R = 817.0
γ = 0.333 (болезнь = 3.0 дн): Пик I = 182.0, Время пика = 25.1, Финальное R = 680.0
γ = 0.5 (болезнь = 2.0 дн): Пик I = 19.0, Время пика = 25.3, Финальное R = 145.0
γ = 1.0 (болезнь = 1.0 дн): Пик I = 11.0, Время пика = 0.2, Финальное R = 15.0
γ = 2.0 (болезнь = 0.5 дн): Пик I = 10.0, Время пика = 0.0, Финальное R = 11.0

```

Рисунок 4.6: Анализ разных параметров γ

4.5 4. Детерминированная длительность болезни

Я заменяю экспоненциальное время восстановления на фиксированное значение $1/\gamma$ (рис.7)

```

# Основная логика жизни индивида
@resumable function live(env::ConcurrentSim.Simulation,
individual::SIRPerson, m::SIRModel)
    while individual.status == :S
        @yield timeout(env, rand(Exponential(1/m.c)))
        alter = individual
        while alter == individual
            N = length(m.allIndividuals)
            index = rand(DiscreteUniform(1, N))
            alter = m.allIndividuals[index]
        end
        if alter.status == :I
            if rand(Uniform(0, 1)) < m.β
                individual.status = :I
                infection_update!(env, m)
            end
        end
    end
    if individual.status == :I
        @yield timeout(env, 1/m.γ)
        individual.status = :R
        recovery_update!(env, m)
    end
end
end

```

Рисунок 4.7: фиксированное значение $1/\gamma$

Я сравниваю детерминированную версию со стохастической версией. Что касается стохастической модели, то мы можем видеть, что она немного грубо-вата, поскольку отражает реальность происходящего. Что касается детерминированной модели, то она немного сглажена, поскольку показывает вычислительную модель, основанную на средних значениях и уравнениях. (рис.8)

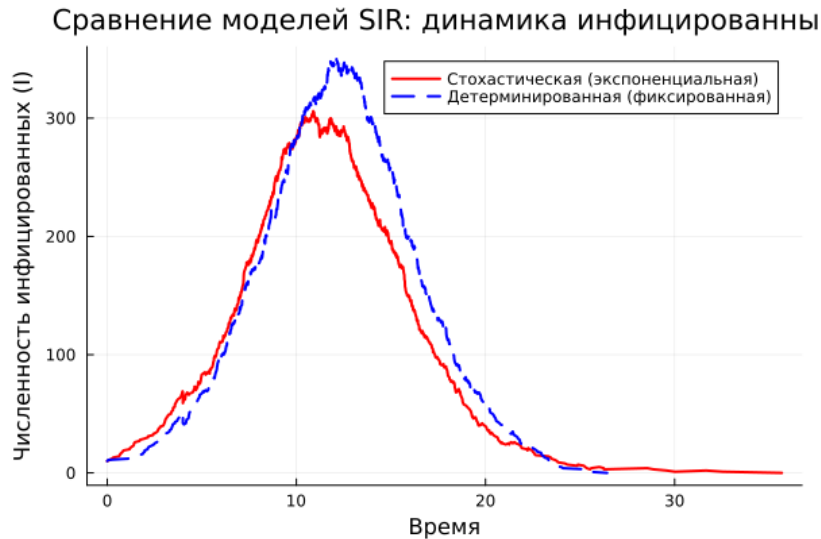


Рисунок 4.8: Сравнение версии

Ниже приведен код, результаты расчетов и графики для кода в файле `scripts/sir_deterministic.qmd`

4.6 Модель SIR

Модель создается, процессы активируются, и запуск выполняется. Результат преобразуется во фрейм данных. График с кривыми S, I, R создается и сохраняется

```
using DrWatson
@quickactivate "project"
include(sourcedir("Sir_model.jl"))
include(sourcedir("Sir_deterministic.jl"))
using Random, StatsPlots, BenchmarkTools
gr(fmt=:png)
```

```
└ Warning: DrWatson could not find a project file by recursively checking given `dir`
  | (given dir: /home/nelianjovu)
```


└─ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84

Plots.GRBackend()

Параметры модели

```
tmax = 40.0
u0 = [990, 10, 0] # S, I, R
p = [0.05, 10.0, 0.25] #  $\beta$ , c,  $\gamma$ 

Random.seed!(1234)
```

TaskLocalRNG()

Запуск модели

```
stoch_model = MakeSIRModelD(u0, p)
activate(stoch_model)
sir_run(stoch_model, tmax)
data_stoch = out(stoch_model)

deter_model = MakeSIRModelD(u0, p)
activate(deter_model)
sir_run(deter_model, tmax)
data_deter = out(deter_model)
```

	t	S	I	R
	Float64	Int64	Int64	Int64
1	0.0	990	10	0
2	0.0550178	989	11	0
3	1.08959	988	12	0
4	1.27335	987	13	0
5	1.64664	986	14	0
6	1.67492	985	15	0
7	1.87887	984	16	0
8	1.90739	983	17	0
9	1.9808	982	18	0
10	1.98296	981	19	0
11	2.13035	980	20	0
12	2.1582	979	21	0
13	2.19511	978	22	0
14	2.34092	977	23	0
15	2.4615	976	24	0
16	2.50577	975	25	0
17	2.62611	974	26	0
18	2.68531	973	27	0
19	2.80709	972	28	0
20	2.83319	971	29	0
21	2.86673	970	30	0
22	3.02579	969	31	0
23	3.02821	968	32	0
24	3.03356	967	33	0
25	3.2002	966	34	0
26	3.28802	965	35	0
27	3.29785	964	36	0
28	3.38219	963	37	0
29	3.40548	962	38	0 ₁₈
30	3.42886	961	39	0
...	...	...	...	...

Визуализация

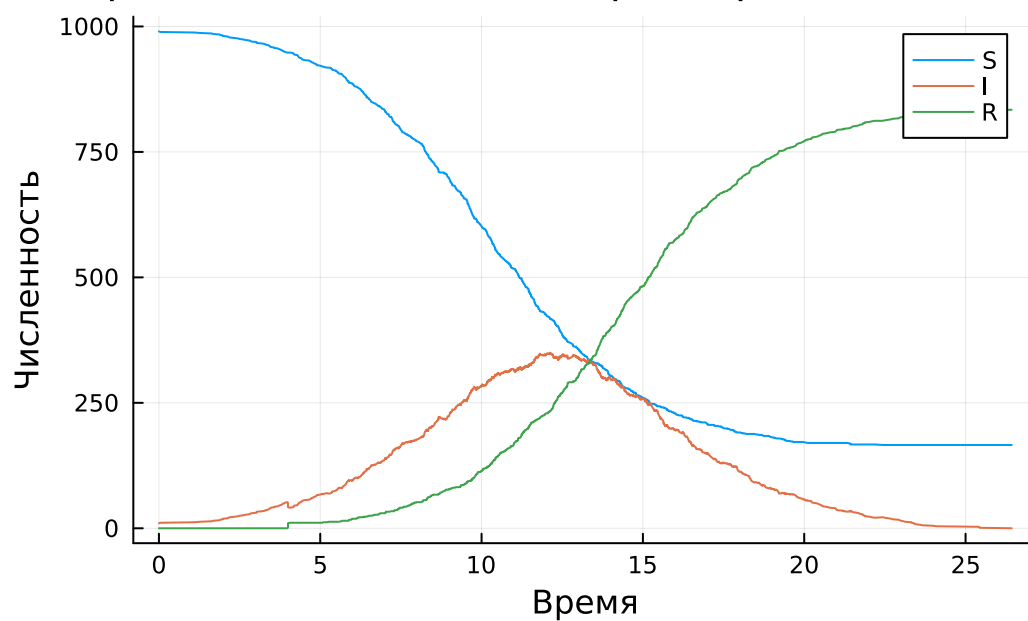
```
plot1 = @df data_deter plot(  
    :t,  
    [:S :I :R],  
    labels = ["S" "I" "R"],  
    xlab = "Время",  
    ylab = "Численность",  
    title = "Детерминированная SIR модель (фиксированное выздоровление)",  
)  
savefig(plotsdir("sir_deter.png"))  
display(plot1)  
  
plot2 = @df data_stoch plot(  
    :t,  
    [:S :I :R],  
    labels = ["S" "I" "R"],  
    xlab = "Время",  
    ylab = "Численность",  
    title = "Стохастическая SIR модель (экспоненциальное выздоровление)",  
)  
savefig(plotsdir("sir_stoch.png"))  
display(plot2)  
  
plot3 = plot(  
    data_stoch.t, data_stoch.I,  
    label = "Стохастическая (экспоненциальная)",  
    xlabel = "Время",  
    ylabel = "Численность инфицированных (I)",
```

```

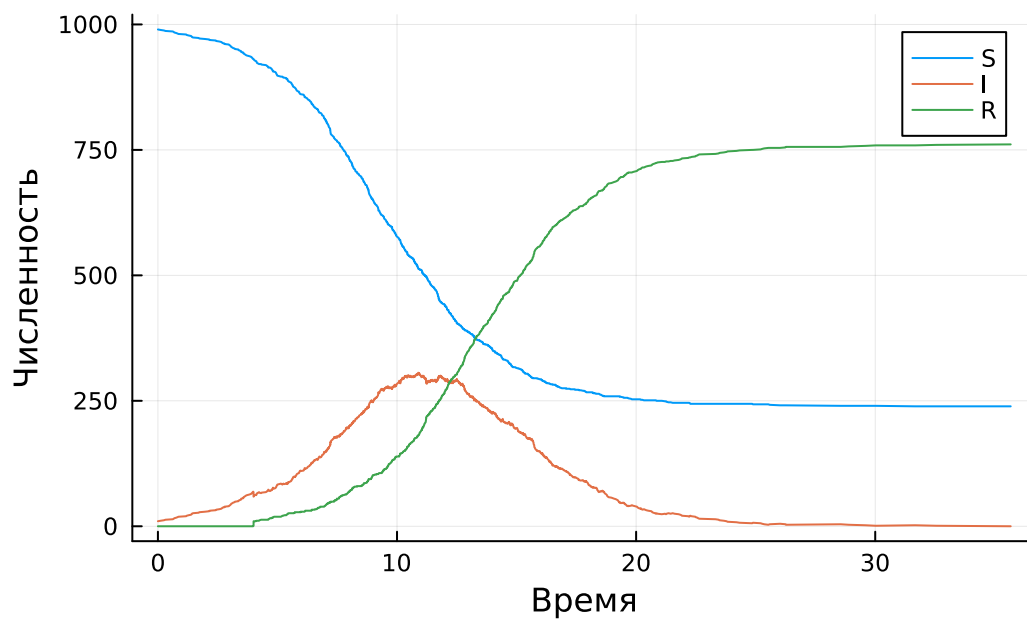
    title = "Сравнение моделей SIR: динамика инфицированных",
    linewidth = 2,
    color = :red,
    legend = :topright,
    grid = true,
)
plot!(
    plot3,
    data_deter.t, data_deter.I,
    label = "Детерминированная (фиксированная)",
    linewidth = 2,
    color = :blue,
)
savefig(plotsdir("sir_comparison.png"))

```

иницированная SIR модель (фиксированное вызд



Стихическая SIR модель (экспоненциальное выздс



"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab08/project/plots/sir_comparison.png"

4.7 5. Оценка производительности

Используя макрос **benchmark**, я измеряю время выполнения `sir_run` для популяции из 10 000 человек(рис.9)

```

Sir_model.jl x  sir_des.jl x  sample.jl x  sir_evaluation.jl x
using DrWatson
@quickactivate "project"
include(scrdir("Sir_model.jl"))
using Random, StatsPlots, BenchmarkTools

# Параметры модели
tmax = 40.0
u0 = [9990, 10, 0] # S, I, R
p = [0.05, 10.0, 0.25] #  $\beta$ ,  $c$ ,  $\gamma$ 

Random.seed!(1234)

# Запуск модели
des_model = MakeSIRModel(u0, p)
activate(des_model)

bench_results = @benchmark sir_run($des_model, $tmax) samples = 10
evals = 1

# Результат
println("\nРезультаты бенчмаркинга для N=10 000:")
println(" Минимальное время: $(round(minimum(bench_result).time /
1e9, digits=4)) секунд")
println(" Медианное время: $(round(median(bench_result).time /
1e9, digits=4)) секунд")
println(" Среднее время: $(round(mean(bench_result).time / 1e9,
digits=4)) секунд")
println(" Максимальное время: $(round(maximum(bench_result).time /
1e9, digits=4)) секунд")

```

Рисунок 4.9: Измерение время выполнения программа

Модель была протестирована на популяции из 10 000 особей в течение 40 единиц времени. Моделирование завершилось в среднем менее чем за 0,01 секунды, продемонстрировав, что реализация на основе ConcurrentSim является высокоэффективной для популяций такого размера(рис.10)

```

nelianjovu@Nebby:~/work/study/2026-1/2026-1==study--simulationmod/2026-1--study-
simulationmod/labs/lab08/project$ julia --project=. scripts/sir_evaluation.jl

Результаты бенчмаркинга для N=10 000:
Минимальное время: 0.0 секунд
Медианное время: 0.0 секунд
Среднее время: 0.0 секунд
Максимальное время: 0.0001 секунд

```

Рисунок 4.10: Время выполнения программа

4.8 6. Сохранение результатов в CSV

Я добавляю автоматическое сохранение итоговой таблицы в каталог data/sims/ с уникальным именем, содержащим параметры запуска(рис.11)

```
filename = "sir_$(u0[1])_$(u0[2])_$(p[1])_$(p[2])_$(p[3]).csv"
CSV.write(datadir("sims", filename), data_des)
```

Рисунок 4.11: сохранение итоговой таблицы

Код сработал, и файл был сохранен(рис.12)

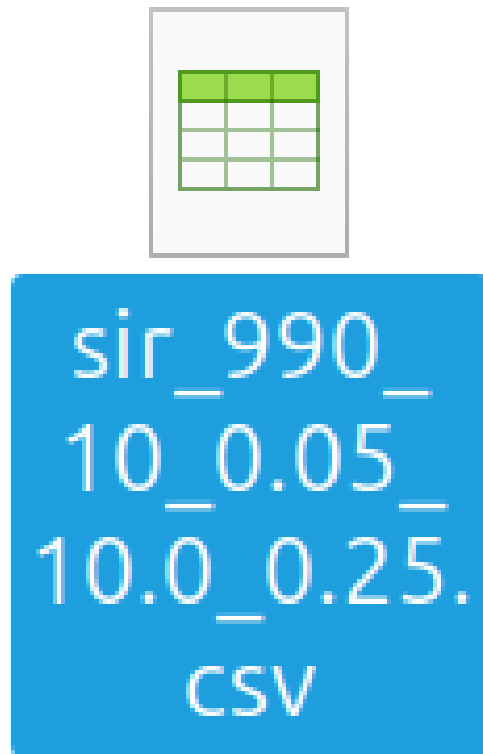


Рисунок 4.12: сохранение итоговой таблицы

4.9 7. Добавление демографических событий

Я расширяю модель, добавляя к жизненному циклу индивидуума возможность смерти (с постоянной интенсивностью μ) и рождения новых восприим-

чивых особей. Смерть удаляет индивидуума из популяции, рождение добавляет нового в состояние : S.(рис 13)

```

*sample.jl x sir_evaluation.jl x Sir_model_1.jl x sir_demographic.jl x
@yield timeout(env, Inf)
end
end
end
end
end

@resumable function birth_process(env::ConcurrentSim.Simulation,
m::SIRModel)
while true
if m.μ > 0
@yield timeout(env, rand(Exponential(1/m.μ)))
new_id = length(m.allIndividuals) + 1
new_individual = SIRPerson(new_id, :S)
push!(m.allIndividuals, new_individual)
birth_update!(env, m)
@process live(env, new_individual, m)
else
@yield timeout(env, Inf)
end
end
end
end

```

Рисунок 4.13: добавление возможности смерти и рождения

В отличие от стандартной модели SIR, в модели с демографией количество выздоровевших людей увеличивается, а затем снова начинает уменьшаться. Это связано с тем, что выздоровевшие люди могут умереть от естественных причин(рис.14)

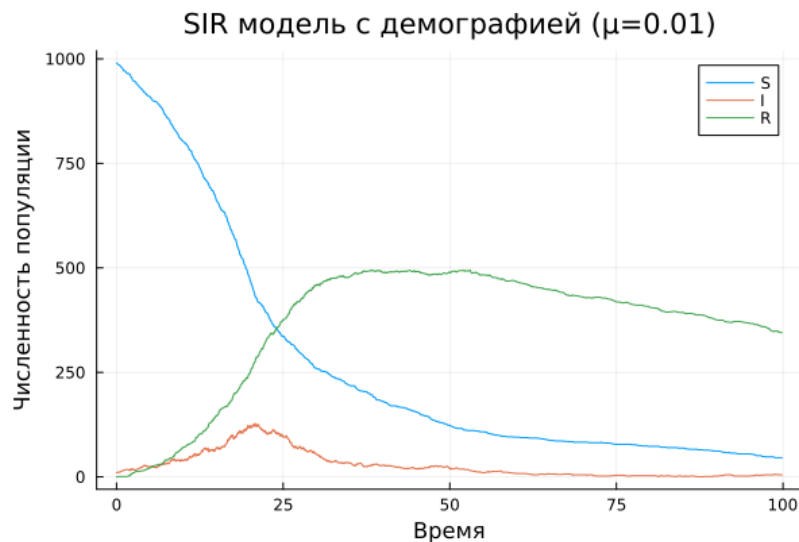


Рисунок 4.14: добавление возможности смерти и рождения

Ниже приведен код, результаты расчетов и графики для кода в файле `scripts/sir_demographic.qmd`

4.10 Модель SIR

Модель создается, процессы активируются, и запуск выполняется. Результат преобразуется во фрейм данных. График с кривыми S, I, R создается и сохраняется

```
using DrWatson
@quickactivate "project"

if !isdefined(Main, :SIRPerson)
    include(sourcedir("Sir_model_1.jl"))
end

using StatsPlots
gr(fmt=:png)
```

```
└ Warning: DrWatson could not find find a project file by recursively checking given `dir`
  | (given dir: /home/nelianjovu)
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
```

Plots.GRBackend()

Параметры

```
p = [0.05, 10.0, 0.25, 0.01]
u0 = [990, 10, 0]
model = MakeSIRModel(u0, p)
```

```
activate(model)
sir_run(model, 100.0)

results = out(model)
```

	t	S	I	R
	Float64	Int64	Int64	Int64
1	0.0	990	10	0
2	0.00934807	989	11	0
3	0.447958	988	12	0
4	0.832778	987	13	0
5	0.926698	986	14	0
6	1.24744	985	15	0
7	1.49851	984	16	0
8	1.54501	983	17	0
9	1.59558	982	18	0
10	1.64916	981	19	0
11	1.6773	980	20	0
12	1.8289	979	21	0
13	1.86758	978	22	0
14	1.9757	977	23	0
15	1.99248	976	24	0
16	2.0526	975	25	0
17	2.11483	974	26	0
18	2.17464	973	27	0
19	2.18082	972	28	0
20	2.21973	971	29	0
21	2.23986	970	30	0
22	2.25402	969	31	0
23	2.39795	968	32	0
24	2.4003	967	33	0
25	2.41576	966	34	0
26	2.49732	965	35	0
27	2.50687	964	36	0
28	2.5274	963	37	0
29	2.57474	962	38	0
30	2.61118	961	39	0
...	...	...	...	...

Визуализация

```
plot(results.t, [results.S results.I results.R],  
      label=["S" "I" "R"],  
      xlabel="Время", ylabel="Численность популяции",  
      title="SIR модель с демографией ( $\beta = p[4]$ )")  
savefig(plotsdir("sir_with_demography.png"))  
  
println("Готово!  $\beta =$ ", p[4])
```

Готово! $\beta = 0.01$

4.11 8. Вакцинация

Я внедряю стратегию вакцинации(рис.15)

```

Sir_model_1.jl x sir_demographic.jl x *Sir_model_vacc.jl x
increment!(m.Ra)
end

function batch_vaccination_update!(sim::ConcurrentSim.Simulation,
m::SIRModel)
    push!(m.ta, ConcurrentSim.now(sim))
    current_S = count(ind.status == :S for ind in m.allIndividuals)
    current_I = count(ind.status == :I for ind in m.allIndividuals)
    current_R = count(ind.status == :R for ind in m.allIndividuals)
    push!(m.Sa, current_S)
    push!(m.Ia, current_I)
    push!(m.Ra, current_R)
end

@resumable function vaccinate(env::ConcurrentSim.Simulation,
m::SIRModel, time::Float64, fraction::Float64)
    @yield timeout(env, time)

    susceptible_ids = [i for (i, ind) in enumerate(m.allIndividuals)
if ind.status == :S]
    num_to_vaccinate = Int(round(length(susceptible_ids) *
fraction))

    if num_to_vaccinate > 0
        to_vaccinate = rand(m.rng, susceptible_ids,
num_to_vaccinate, replace=false)
        for id in to_vaccinate
            m.allIndividuals[id].status = :R
        end
        batch_vaccination_update!(env, m)
    end
end
end

```

Рисунок 4.15: Вакцинация

На графике мы видим, что в определенный момент времени некоторые из восприимчивых мгновенно переносят R. Для предотвращения эпидемии необходимо вакцинировать около 50% населения(рис.16)

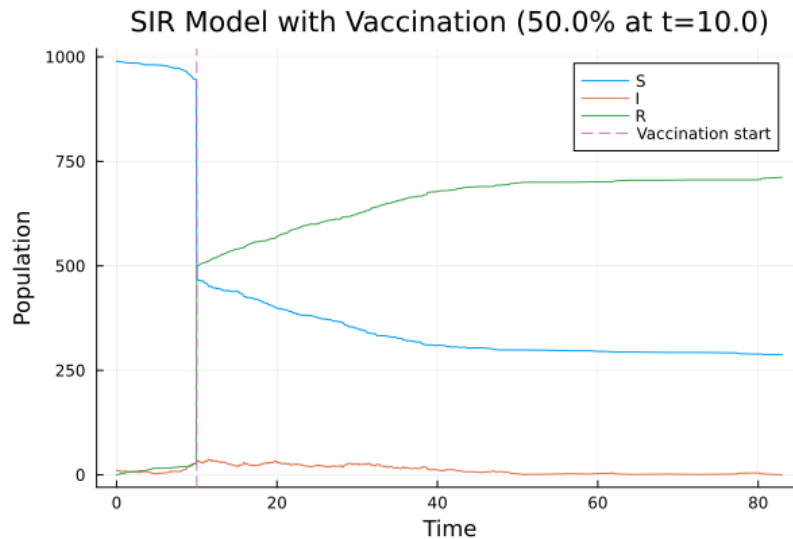


Рисунок 4.16: График вакцинации

4.12 9. Модель SEIR

Разница между sir и $seir$ в том, что для sir это восприимчивость - заражение - выздоровление. Как только вы заразитесь, вы можете заразить других. В модели $seir$ есть дополнительный этап, который заключается в e - воздействии, в этом случае, как только вы подвергаетесь воздействию, требуется время, чтобы вы заразились и были восстановлены.

Я ввожу период ожидания (статус :E)(рис.17)

```

sir_des.jl  x  Sir_model_vacc.jl  x  sir_vacc.jl  x  Seir.jl  x  seir.jl  x
using ResumableFunctions, ConcurrentSim, Distributions, DataFrames,
Random

# Вспомогательные функции для обновления массивов состояния
function increment!(a::Array{Int64})
    push!(a, a[length(a)] + 1)
end
function decrement!(a::Array{Int64})
    push!(a, a[length(a)] - 1)
end
function carryover!(a::Array{Int64})
    push!(a, a[length(a)])
end

# Структуры данных для SEIR
mutable struct SEIRPerson
    id::Int64
    status::Symbol # :S, :E, :I, :R
end

mutable struct SEIRModel
    sim::ConcurrentSim.Simulation
    β::Float64
    c::Float64
    γ::Float64
    σ::Float64
    ta::Array{Float64}
    Sa::Array{Int64}
    Ea::Array{Int64}
    Ia::Array{Int64}
end

```

Рисунок 4.17: Модель SEIR

При тех же параметрах, что и в модели sir, болезнь распространяется гораздо медленнее, и в модели seir эпидемии практически отсутствуют.(рис.18)

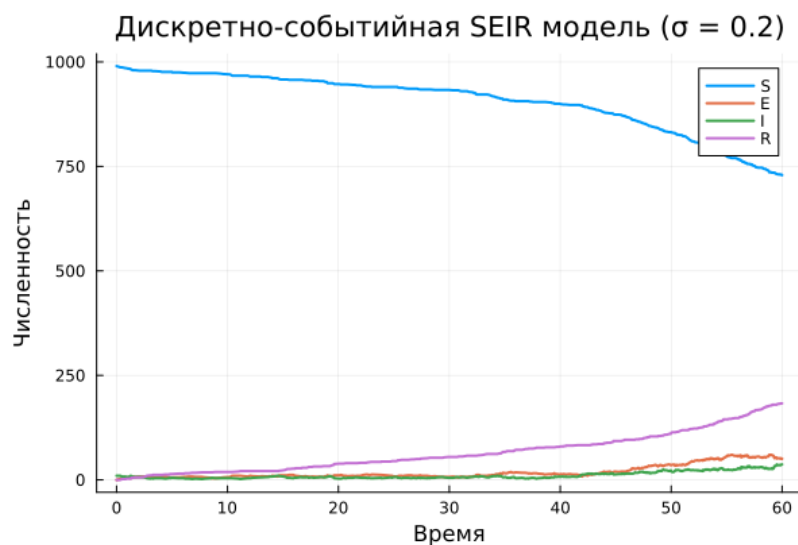


Рисунок 4.18: График модели SEIR

Ниже приведен код, результаты расчетов и графики для кода в файле `scripts/seir.qmd`

Пример использования модели SEIR

```
using DrWatson
@quickactivate "project"
include(scrdir("Seir.jl"))
using Random, StatsPlots, BenchmarkTools, CSV, Dates
gr(fmt=:png)
```

```
└ Warning: DrWatson could not find find a project file by recursively checking given `dir`
  | (given dir: /home/nelianjovu)
  └ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
```

Plots.GRBackend()

Параметры модели SEIR

```
tmax = 60.0
u0 = [990, 0, 10, 0] # S, E, I, R
p = [0.05, 10.0, 0.25] #  $\beta$ ,  $c$ ,  $\gamma$ 
 $\gamma$  = 0.2 # Скорость перехода из E в I (1/средняя длительность латентного периода)

Random.seed!(1234)
```

TaskLocalRNG()

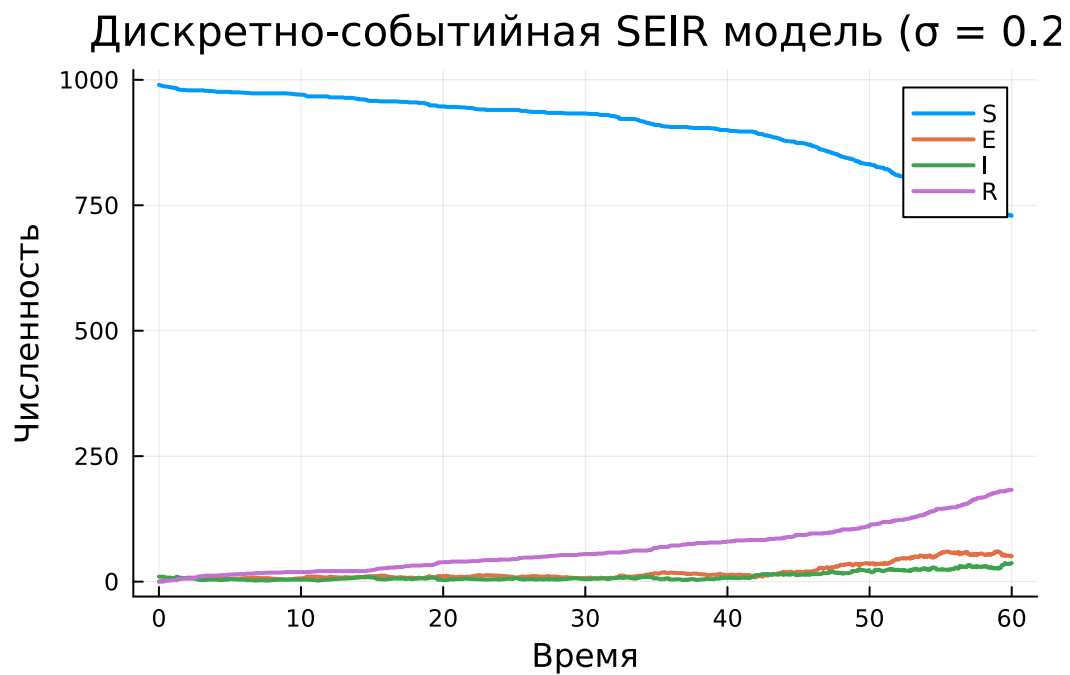
Запуск SEIR модели


```
seir_model = MakeSEIRModel(u0, p, □)
activate_seir(seir_model)
seir_run(seir_model, tmax)
data_seir = out_seir(seir_model)
```

	t	S	E	I	R
	Float64	Int64	Int64	Int64	Int64
1	0.0	990	0	10	0
2	0.110851	989	1	10	0
3	0.24955	988	2	10	0
4	0.315197	987	3	10	0
5	0.418738	987	3	9	1
6	0.457041	987	3	8	2
7	0.658997	986	4	8	2
8	0.812194	985	5	8	2
9	0.913646	985	5	7	3
10	1.00373	984	6	7	3
11	1.16735	984	5	8	3
12	1.18218	983	6	8	3
13	1.2699	983	5	9	3
14	1.27274	983	4	10	3
15	1.28548	982	5	10	3
16	1.30828	982	5	9	4
17	1.31811	982	5	8	5
18	1.38263	981	6	8	5
19	1.45015	980	7	8	5
20	1.71622	980	7	7	6
21	2.07612	979	8	7	6
22	2.52487	979	8	6	7
23	2.56655	979	8	5	8
24	2.5969	979	7	6	8
25	2.68616	979	7	5	9
26	2.7201	979	7	4	10
27	3.07698	979	7	3	11
28	3.14162	979	6	4	11
29	3.23597	978	7	4 34	11
30	3.25924	978	7	3	12
...	...	...	...	...	...

Визуализация

```
plot1 = @df data_seir plot(  
    :t,  
    [:S :E :I :R],  
    labels = ["S" "E" "I" "R"],  
    xlab = "Время",  
    ylab = "Численность",  
    title = "Дискретно-событийная SEIR модель ( $\sigma = 0.2$ )",  
    linewidth = 2,  
)  
savefig(plotsdir("seir_des.png"))  
display(plot1)
```



5 Выводы

В этой лабораторной работе, я изучила дискретно-событийный подход к моделированию на примере классической модели распространения инфекции SIR, как реализовать стохастическую дискретно-событийную модель в Julia и как анализировать влияние параметров, сравнивать их со стохастическими и детерминированными версиями, оценивать производительность и модифицировать систему

Список литературы

1. Кулябов Д.С. simulation-modeling-lab.pdf. 8 Реализация основных моделей в дискретно-событийном подходе. [Электронный ресурс]